TEAMBOOK

Matemáticas para Programación Competitiva

Fórmulas, conteo, combinatoria, probabilidad y teoría de números

---

Manual de consulta rápida para resolver problemas

Enfoque práctico con notación matemática y plantillas en C++17

Material de trabajo del equipo

Versión 1.0 – noviembre de 2026

Cómo usar este teambook

Este documento está pensado para acompañar la resolución de problemas, no para reemplazar un curso completo de matemática discreta. La prioridad es reconocer qué fórmula corresponde, comprobar sus condiciones y llevarla a una implementación segura.

- **Primero identifica el modelo.** ¿Se cuentan elecciones, se trabaja con restos, se necesita una probabilidad, una suma o una relación entre puntos?
- **Después revisa las condiciones.** Muchas fórmulas cambian si los objetos son distinguibles, si se permite repetir, si el módulo es primo o si el orden importa.
- **Por último piensa en los límites.** Una fórmula correcta puede fallar por desbordamiento, división modular inválida o complejidad demasiado alta.

Ojo en competencia

No memoricen una fórmula sin su condición. Por ejemplo, dividir por $k!$ en módulo M solo es válido cuando $k!$ tiene inverso módulo M. Si M es primo, esto se cumple cuando $k < M$.

Convenciones del documento. Usaremos $n \geq 0$ salvo que se indique lo contrario; los arreglos suelen indexarse desde cero en C++; los módulos se suponen positivos; y las complejidades se expresan respecto al tamaño de la entrada. Para enteros grandes se recomienda `long long` o `__int128` según los límites.

Relación con las referencias. La organización sigue el enfoque práctico de *Competitive Programming* de Steven y Felix Halim (volúmenes 1 y 2), complementado conceptualmente con los temas de teoría de números, combinatoria, álgebra y geometría que se documentan en CP-Algorithms. Las fórmulas se presentan como una hoja de trabajo independiente y no como una transcripción de esos libros.

Índice general

1. Hoja de consulta rápida **1**

1.1. Fórmulas que aparecen constantemente . . . 1

1.2. Elección rápida de técnica . . . 1

2. Álgebra, potencias, logaritmos y sumatorias **3**

2.1. Potencias y raíces . . . 3

2.2. Logaritmos . . . 3

2.3. Sumas básicas . . . 3

2.4. Progresiones . . . 4

2.4.1. Aritmética . . . 4

2.4.2. Geométrica . . . 4

2.5. Sumas telescópicas . . . 4

2.6. Sumas armónicas y cotas . . . 4

2.7. Sumas dobles útiles . . . 5

2.8. Identidades y desigualdades . . . 5

3. Divisibilidad y teoría de números **6**

3.1. MCD y MCM . . . 6

3.2. Identidad de Bézout y Euclides extendido . . . 6

3.3. Ecuaciones diofánticas lineales . . . 7

3.4. Números primos y factorización . . . 7

3.4.1. Cantidad y suma de divisores . . . 7

3.4.2. Producto de divisores . . . 7

3.5. Función phi de Euler . . . 7

3.6. Función de Möbius . . . 8

3.7. Conteo de múltiplos . . . 8

3.8. Criba de Eratóstenes . . . 8

3.8.1. SPF: menor factor primo . . . 9

4. Aritmética modular **10**

4.1. Congruencias . . . 10

4.2. Normalización de negativos . . . 10

4.3. Potenciación binaria . . . 10

4.4. Inverso modular . . . 11
4.5. Pequeño teorema de Fermat y teorema de Euler . . . 11
4.6. Teorema chino del resto (CRT) . . . 11
4.7. Teorema de Wilson . . . 12
4.8. Exponente de un primo en un factorial . . . 12
4.9. Teorema de Lucas . . . 12

5. Combinatoria y técnicas de conteo 13
5.1. Las dos preguntas antes de contar . . . 13
5.2. Reglas básicas . . . 13
5.3. Permutaciones y arreglos . . . 13
5.4. Combinaciones . . . 14
5.5. Combinaciones con repetición . . . 14
5.6. Estrellas y barras . . . 14
5.7. Coeficiente multinomial . . . 15
5.8. Teorema del binomio . . . 15
5.9. Identidades de Vandermonde y hockey-stick . . . 15
5.10. Principio de inclusión-exclusión . . . 15
5.11. Lema de Burnside . . . 16
5.12. Desarreglos . . . 16
5.13. Números de Catalan . . . 16
5.14. Números de Stirling . . . 17
5.14.1. Segunda especie . . . 17
5.14.2. Primera especie . . . 17
5.15. Números de Bell . . . 17
5.16. Particiones de enteros . . . 17
5.17. Conteos pequeños muy comunes . . . 18

6. Funciones generadoras y convoluciones 19
6.1. Idea de función generadora . . . 19
6.2. Funciones generadoras básicas . . . 19
6.3. Convolución . . . 19

7. Probabilidad y valor esperado 21
7.1. Probabilidad básica . . . 21
7.2. Variables aleatorias discretas . . . 21
7.3. Variables indicadoras . . . 22
7.4. Varianza . . . 22
7.5. Distribuciones discretas comunes . . . 22
7.6. Problemas clásicos de esperanza . . . 22
7.6.1. Número esperado de éxitos . . . 22
7.6.2. Coleccionista de cupones . . . 23

7.6.3. Tiempo de espera geométrico 23

8. Sucesiones, recurrencias y matrices **24**
8.1. Fibonacci 24
8.2. Recurrencias lineales 24
8.3. Multiplicación de matrices 24
8.4. Fibonacci con matrices 25
8.5. Fast doubling para Fibonacci 25
8.6. Diferencias finitas 25

9. Combinatoria modular y precálculos **26**
9.1. Factoriales e inversos factoriales 26
9.2. Combinaciones con k pequeño 27
9.3. Triángulo de Pascal 27
9.4. Combinaciones cuando el módulo no es primo 27

10. Álgebra lineal **28**
10.1. Vectores y producto punto 28
10.2. Determinante 28
10.3. Sistemas lineales 28
10.4. Rango 29

11. Geometría para programación competitiva **30**
11.1. Puntos y vectores 30
11.2. Producto punto y producto cruzado 30
11.3. Orientación de tres puntos 30
11.4. Área 30
11.5. Rectas 31
11.6. Proyección de un punto sobre una recta 31
11.7. Círculo 31
11.8. Puntos de coordenadas enteras 32
11.9. Teorema de Pick 32
11.10Transformaciones 32

12. Bitmasks, XOR y juegos **33**
12.1. Operaciones de bits 33
12.2. Enumerar submáscaras 33
12.3. XOR 33
12.4. Nim 33
12.5. Grundy y mex 34

13. Transformadas y multiplicación rápida de polinomios **35**
13.1. Convolución discreta 35

13.2. Transformada discreta de Fourier (DFT) . . . 35
13.3. NTT . . . 35

14. Técnicas numéricas y seguridad de implementación **36**
14.1. Tipos enteros . . . 36
14.2. Raíz cuadrada entera . . . 36
14.3. Comparaciones de fracciones . . . 36
14.4. Reducción de fracciones . . . 36
14.5. Errores frecuentes de precisión . . . 36

15. Checklist de resolución en competencia **38**
15.1. Complejidades orientativas . . . 38
15.2. Casos de prueba matemáticos . . . 38

A. Plantillas C++17 de apoyo **40**
A.1. MCD y MCM . . . 40
A.2. Menor factor primo (SPF) con criba lineal . . . 40
A.3. Función phi con factorización . . . 41
A.4. Potenciación de matrices . . . 41
A.5. Orientación geométrica entera . . . 41
A.6. Plantilla mínima de competencia . . . 42

B. Glosario breve **43**

C. Referencias y lecturas para ampliar **44**

Capítulo 1

Hoja de consulta rápida

1.1 Fórmulas que aparecen constantemente

Tema	Fórmula
Suma de los primeros n enteros	$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$
Suma de cuadrados	$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$
Suma de cubos	$\sum_{i=1}^{n} i^3 = \left(\frac{n(n+1)}{2}\right)^2$
Progresión aritmética	$S_n = \frac{n(a_1+a_n)}{2}$
Progresión geométrica	$S_n = a\frac{r^n-1}{r-1}$, si $r \neq 1$
Combinaciones	$\binom{n}{k} = \frac{n!}{k!(n-k)!}$
Permutaciones	$P(n) = n!$
Permutaciones con repetidos	$\frac{n!}{n_1!n_2!\cdots n_t!}$
Estrellas y barras	$\#\{x_i \geq 0 : \sum x_i = n\} = \binom{n+k-1}{k-1}$
Inclusión-exclusión, dos conjuntos	$\lvert A \cup B\rvert = \lvert A\rvert + \lvert B\rvert - \lvert A \cap B\rvert$
Catalan	$C_n = \frac{1}{n+1}\binom{2n}{n}$
Euler / totiente	$\varphi(n) = n\prod_{p\mid n}\left(1-\frac{1}{p}\right)$
Divisores de $n = \prod p_i^{a_i}$	$d(n) = \prod_i (a_i+1)$
Suma de divisores	$\sigma(n) = \prod_i \frac{p_i^{a_i+1}-1}{p_i-1}$
Distancia euclidiana	$d(P,Q) = \sqrt{(x_2-x_1)^2+(y_2-y_1)^2}$
Área de triángulo	$A = \frac{1}{2}\lvert(B-A)\times(C-A)\rvert$
Teorema de Pick	$A = I + \frac{B}{2} - 1$

1.2 Elección rápida de técnica

Pista del enunciado	**Idea que conviene revisar**
"¿De cuántas formas?", elegir k de n	Combinaciones, permutaciones, estrellas y barras.
Contar elementos que cumplen al menos una condición	Inclusión-exclusión.
Muchos restos, divisibilidad o números enormes	Módulo, MCD, inversos, CRT.
Factorizar muchos números hasta N	Criba, SPF y factorización rápida.
Suma de una función sobre divisores	Factorización, φ, Möbius o precálculo.
Probabilidad de varios eventos independientes	Producto de probabilidades; distribución binomial.
Cantidad esperada de elementos/eventos	Variables indicadoras y linealidad de esperanza.
Una relación depende de los términos anteriores	Recurrencia, programación dinámica, matrices.
Pares de puntos, área, orientación o intersección	Geometría vectorial y producto cruzado.
Juego por turnos con montones independientes	Nim, XOR y Grundy.

Capítulo 2

Álgebra, potencias, logaritmos y sumatorias

2.1 Potencias y raíces

Para $a \neq 0$ y exponentes enteros compatibles:

$$a^m a^n = a^{m+n}, \qquad \frac{a^m}{a^n} = a^{m-n},$$
$$(a^m)^n = a^{mn}, \qquad (ab)^n = a^n b^n,$$
$$a^0 = 1, \qquad a^{-n} = \frac{1}{a^n}.$$

Si $a > 0$, entonces $a^{m/n} = \sqrt[n]{a^m}$. En programación, una raíz entera debe calcularse con cuidado: `sqrt` usa punto flotante y puede quedar a una unidad del resultado exacto para números grandes.

2.2 Logaritmos

Para bases positivas a, b, c con $a, b \neq 1$:

$$\log_a(xy) = \log_a x + \log_a y,$$
$$\log_a(x/y) = \log_a x - \log_a y,$$
$$\log_a(x^r) = r \log_a x,$$
$$\log_a x = \frac{\log_b x}{\log_b a}.$$

En análisis de algoritmos, las bases se diferencian por una constante multiplicativa: $\log_2 n = \Theta(\log_{10} n)$.

Idea para reconocer el problema

Si un proceso divide el tamaño entre 2 repetidamente, suele tener $\mathcal{O}(\log n)$ pasos. Si prueba todos los divisores hasta $\sqrt{n}$, realiza $\mathcal{O}(\sqrt{n})$ iteraciones.

2.3 Sumas básicas

Para $n \geq 1$:

$$\sum_{i=1}^{n} 1 = n, \tag{2.1}$$

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}, \tag{2.2}$$

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}, \tag{2.3}$$

$$\sum_{i=1}^{n} i^3 = \left(\frac{n(n+1)}{2}\right)^2, \tag{2.4}$$

$$\sum_{i=1}^{n} (2i-1) = n^2, \tag{2.5}$$

$$\sum_{i=1}^{n} 2i = n(n+1). \tag{2.6}$$

2.4 Progresiones

2.4.1 Aritmética

Si $a_i = a_1 + (i-1)d$, entonces:

$$a_n = a_1 + (n-1)d, \qquad S_n = \frac{n(a_1+a_n)}{2} = \frac{n}{2}(2a_1 + (n-1)d).$$

2.4.2 Geométrica

Si $a_i = ar^{i-1}$:

$$S_n = a + ar + \cdots + ar^{n-1} = \begin{cases} a\dfrac{r^n - 1}{r-1}, & r \neq 1, \\ na, & r = 1. \end{cases}$$

Si $|r| < 1$, la suma infinita converge a $S = \frac{a}{1-r}$.

2.5 Sumas telescópicas

Una suma telescópica aparece cuando cada término cancela parte del siguiente.

$$\sum_{k=1}^{n} \frac{1}{k(k+1)} = \sum_{k=1}^{n} \left(\frac{1}{k} - \frac{1}{k+1}\right) = 1 - \frac{1}{n+1},$$

$$\sum_{k=1}^{n} (f(k) - f(k+1)) = f(1) - f(n+1).$$

2.6 Sumas armónicas y cotas

$$H_n = \sum_{k=1}^{n} \frac{1}{k} = \ln n + \gamma + \mathcal{O}(1/n),$$

donde $\gamma \approx 0.57721$ es la constante de Euler-Mascheroni. Por eso, algoritmos que visitan todos los múltiplos de cada i suelen acumular un costo $\mathcal{O}(n \log n)$.

2.7 Sumas dobles útiles

$$\sum_{i=1}^{n}\sum_{j=1}^{m} 1 = nm,$$
$$\sum_{i=1}^{n}\sum_{j=1}^{i} 1 = \frac{n(n+1)}{2},$$
$$\sum_{i=1}^{n} i(n-i) = \frac{n(n-1)(n+1)}{6}.$$

La última expresión también cuenta pares de posiciones separadas dentro de una secuencia, según cómo se modele el problema.

2.8 Identidades y desigualdades

$$(a+b)^2 = a^2 + 2ab + b^2,$$
$$(a-b)^2 = a^2 - 2ab + b^2,$$
$$a^2 - b^2 = (a-b)(a+b),$$
$$a^3 - b^3 = (a-b)(a^2+ab+b^2),$$
$$a^3 + b^3 = (a+b)(a^2-ab+b^2).$$

Para $x_i \in \mathbb{R}$:

$$\text{AM-GM: } \frac{x_1 + \cdots + x_n}{n} \geq \sqrt[n]{x_1 x_2 \cdots x_n} \quad (x_i \geq 0),$$
$$\text{Cauchy-Schwarz: } \left(\sum a_i^2\right)\left(\sum b_i^2\right) \geq \left(\sum a_i b_i\right)^2.$$

Estas desigualdades pueden servir para demostrar cotas, descartar respuestas o encontrar mínimos/máximos.

Capítulo 3

Divisibilidad y teoría de números

3.1 MCD y MCM

El máximo común divisor se calcula con el algoritmo de Euclides:

$$\gcd(a,b) = \gcd(b, a \bmod b), \qquad \gcd(a,0) = |a|\,.$$

El mínimo común múltiplo satisface:

$$\operatorname{mcm}(a,b) = \frac{|ab|}{\gcd(a,b)}.$$

En código, conviene dividir antes de multiplicar para reducir el riesgo de desbordamiento: `a / gcd(a,b) * b`.

3.2 Identidad de Bézout y Euclides extendido

Existen enteros x, y tales que:

$$ax + by = \gcd(a,b).$$

El algoritmo extendido encuentra esos coeficientes. Es fundamental para inversos modulares y ecuaciones diofánticas.

```
using int64 = long long;

int64 extgcd(int64 a, int64 b, int64 &x, int64 &y) {
    if (b == 0) {
        x = (a >= 0 ? 1 : -1);
        y = 0;
        return (a >= 0 ? a : -a);
    }
    int64 x1, y1;
    int64 g = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}
```

3.3 Ecuaciones diofánticas lineales

La ecuación $ax + by = c$ tiene soluciones enteras si y solo si $\gcd(a, b) \mid c$. Si (x_0, y_0) es una solución y $g = \gcd(a, b)$, todas las soluciones son:

$$x = x_0 + \frac{b}{g}t, \qquad y = y_0 - \frac{a}{g}t, \qquad t \in \mathbb{Z}.$$

Los signos de a y b deben tratarse con cuidado en la implementación.

3.4 Números primos y factorización

Un entero $n > 1$ es primo si sus únicos divisores positivos son 1 y n. Si n es compuesto, tiene algún divisor primo $p \leq \sqrt{n}$.

$$n = \prod_{i=1}^{t} p_i^{a_i}$$

es la factorización prima única de n.

Idea para reconocer el problema

Para una sola consulta, probar divisores hasta $\sqrt{n}$ suele ser suficiente. Para muchas consultas hasta un límite N, usar una criba. Si se requiere factorizar muchos números, precalcular el menor factor primo (SPF).

3.4.1 Cantidad y suma de divisores

Si $n = \prod p_i^{a_i}$:

$$d(n) = \prod_{i=1}^{t}(a_i + 1), \qquad \sigma(n) = \prod_{i=1}^{t}(1 + p_i + \cdots + p_i^{a_i}) = \prod_{i=1}^{t}\frac{p_i^{a_i+1} - 1}{p_i - 1}.$$

La suma de los divisores propios positivos es $s(n) = \sigma(n) - n$.

3.4.2 Producto de divisores

Para $n \geq 1$:

$$\prod_{d|n} d = n^{d(n)/2}.$$

El exponente puede ser semientero en apariencia, pero el resultado es entero; es más seguro razonar por parejas d y n/d.

3.5 Función phi de Euler

$\varphi(n)$ cuenta los enteros de 1 a n que son coprimos con n. Si la factorización prima es conocida:

$$\boxed{\varphi(n) = n \prod_{p|n}\left(1 - \frac{1}{p}\right)}.$$

Propiedades importantes:

$$\varphi(p^k) = p^k - p^{k-1},$$

$$\sum_{d|n} \varphi(d) = n,$$

$$a^{\varphi(m)} \equiv 1 \pmod{m} \quad \text{si } \gcd(a, m) = 1.$$

La última es el teorema de Euler.

3.6 Función de Möbius

La función $\mu(n)$ se define así:

$$\mu(n) = \begin{cases} 1, & n = 1, \\ 0, & n \text{ tiene un factor primo al cuadrado}, \\ (-1)^k, & n \text{ es producto de } k \text{ primos distintos}. \end{cases}$$

Identidad fundamental:

$$\sum_{d|n} \mu(d) = \begin{cases} 1, & n = 1, \\ 0, & n > 1. \end{cases}$$

Inversión de Möbius: si $F(n) = \sum_{d|n} f(d)$, entonces

$$f(n) = \sum_{d|n} \mu(d) F(n/d).$$

Se usa para pasar de contar objetos con una propiedad a contar objetos primitivos, coprimos o sin factores comunes.

3.7 Conteo de múltiplos

La cantidad de múltiplos positivos de k que no superan n es $\lfloor n/k \rfloor$. Por tanto:

$$\sum_{i=1}^{n} \left\lfloor \frac{n}{i} \right\rfloor$$

cuenta pares (i, j) positivos con $ij \leq n$. Esta suma aparece en problemas de divisores y se puede agrupar por cocientes iguales para mejorar la complejidad.

3.8 Criba de Eratóstenes

Para obtener todos los primos hasta N:

1. Marcar inicialmente todos los enteros como posibles primos.
2. Para cada p desde 2 mientras $p^2 \leq N$, si sigue marcado, tachar $p^2, p^2 + p, \ldots$.
3. Los que permanecen marcados son primos.

Complejidad: $\mathcal{O}(N \log \log N)$ tiempo y $\mathcal{O}(N)$ memoria.

```
vector<bool> isPrime(N + 1, true);
isPrime[0] = isPrime[1] = false;
for (int p = 2; 1LL * p * p <= N; ++p) {
    if (!isPrime[p]) continue;
    for (long long j = 1LL * p * p; j <= N; j += p)
        isPrime[(int)j] = false;
}
```

3.8.1 SPF: menor factor primo

Con SPF se puede factorizar cada $x \leq N$ dividiendo repetidamente por `spf[x]`. Una criba lineal permite calcular SPF en $\mathcal{O}(N)$, aunque la criba clásica suele bastar para límites moderados.

Capítulo 4

Aritmética modular

4.1 Congruencias

$$a \equiv b \pmod{m} \iff m \mid (a-b).$$

Se puede sumar, restar y multiplicar congruencias:

$$a \equiv b \pmod{m} \Rightarrow a + c \equiv b + c \pmod{m},$$
$$a \equiv b \pmod{m} \Rightarrow ac \equiv bc \pmod{m}.$$

No se puede cancelar un factor c de forma automática. La cancelación es válida si $\gcd(c, m) = 1$.

4.2 Normalización de negativos

El resto de C++ puede ser negativo. Para representar siempre un resultado entre 0 y $m-1$:

$$\text{norm}(x, m) = ((x \bmod m) + m) \bmod m.$$

4.3 Potenciación binaria

Para calcular $a^n \bmod m$ en $\mathcal{O}(\log n)$:

```
long long modpow(long long a, long long e, long long mod) {
    long long r = 1 % mod;
    a %= mod;
    while (e > 0) {
        if (e & 1) r = (long long)((__int128)r * a % mod);
        a = (long long)((__int128)a * a % mod);
        e >>= 1;
    }
    return r;
}
```

`__int128` es una extensión habitual de GCC/Clang; no forma parte del estándar C++17, pero es útil en jueces que usan esos compiladores. Si el módulo y los operandos son pequeños, `long long` puede ser suficiente.

4.4 Inverso modular

El inverso de a módulo m es un número x que cumple:

$$ax \equiv 1 \pmod{m}.$$

Existe si y solo si $\gcd(a, m) = 1$. Si $m = p$ es primo y $p \nmid a$, por Fermat:

$$a^{-1} \equiv a^{p-2} \pmod{p}.$$

En general, el algoritmo extendido de Euclides obtiene el inverso.

```
long long modInverse(long long a, long long mod) {
    long long x, y;
    long long g = extgcd(a, mod, x, y);
    if (g != 1) return -1; // no existe inverso
    return (x % mod + mod) % mod;
}
```

Ojo en competencia

La división modular no es división entera: $a/b \pmod{m}$ significa $a \cdot b^{-1} \pmod{m}$. Solo está definida cuando b tiene inverso módulo m.

4.5 Pequeño teorema de Fermat y teorema de Euler

Si p es primo y $p \nmid a$:

$$a^{p-1} \equiv 1 \pmod{p}.$$

Si $\gcd(a, m) = 1$:

$$a^{\varphi(m)} \equiv 1 \pmod{m}.$$

Fermat es el caso particular de Euler cuando $m = p$ es primo.

Si $\gcd(a, m) = 1$, el exponente se puede reducir módulo $\varphi(m)$ al calcular $a^e \bmod m$. Si la base no es coprima con el módulo, no se debe reducir el exponente automáticamente: hay que analizar los factores comunes y el tamaño del exponente.

4.6 Teorema chino del resto (CRT)

Para módulos coprimos dos a dos $m_1, \ldots, m_k$, el sistema

$$x \equiv a_i \pmod{m_i}$$

tiene una única solución módulo $M = \prod_i m_i$. Para dos congruencias, con $m_1 \perp m_2$:

$$x = a_1 + m_1 \left((a_2 - a_1) m_1^{-1} \bmod m_2 \right),$$

y se normaliza el resultado módulo $m_1 m_2$.

Para dos módulos no necesariamente coprimos, existe solución si y solo si:

$$a_1 \equiv a_2 \pmod{\gcd(m_1, m_2)}.$$

Cuando existe, la solución es única módulo $\operatorname{mcm}(m_1, m_2)$.

4.7 Teorema de Wilson

Para $p > 1$:

$$p \text{ es primo } \iff (p-1)! \equiv -1 \pmod{p}.$$

Es una caracterización teórica; normalmente no se usa para probar primalidad porque calcular el factorial es costoso.

4.8 Exponente de un primo en un factorial

La cantidad de veces que el primo p aparece en la factorización de $n!$ es:

$$v_p(n!) = \left\lfloor \frac{n}{p} \right\rfloor + \left\lfloor \frac{n}{p^2} \right\rfloor + \left\lfloor \frac{n}{p^3} \right\rfloor + \cdots .$$

Se detiene cuando la potencia supera n. Para un binomial:

$$v_p\left(\binom{n}{k}\right) = v_p(n!) - v_p(k!) - v_p((n-k)!).$$

Si se pide la cantidad de ceros finales de $n!$ escrito en una base b, factoriza $b = \prod p_i^{e_i}$ y usa:

$$\boxed{Z_b(n!) = \min_i \left\lfloor \frac{v_{p_i}(n!)}{e_i} \right\rfloor}.$$

Para base 10, basta tomar $\min(v_2(n!), v_5(n!))$. Sirve para conocer cuántas veces un primo divide a un factorial o coeficiente binomial, y para problemas de ceros finales.

4.9 Teorema de Lucas

Sea p primo y escribamos las representaciones en base p:

$$n = n_0 + n_1 p + \cdots + n_r p^r, \quad k = k_0 + k_1 p + \cdots + k_r p^r.$$

Entonces:

$$\boxed{\binom{n}{k} \equiv \prod_{i=0}^{r} \binom{n_i}{k_i} \pmod{p}.}$$

Si algún $k_i > n_i$, el resultado es cero. Lucas permite calcular binomiales con n grande módulo un primo pequeño.

Capítulo 5

Combinatoria y técnicas de conteo

5.1 Las dos preguntas antes de contar

Antes de elegir una fórmula, respondan:

1. **¿Importa el orden?** Si cambiar el orden produce un resultado distinto, se cuentan arreglos o permutaciones. Si no, se cuentan combinaciones.
2. **¿Se puede repetir?** La repetición cambia el modelo y la fórmula.

También hay que distinguir entre objetos diferentes y objetos idénticos.

5.2 Reglas básicas

Regla de la suma. Si una opción puede realizarse de a formas y otra, incompatible con la primera, de b formas, hay $a + b$ formas.

Regla del producto. Si una tarea tiene etapas con $a_1, a_2, \ldots, a_k$ posibilidades independientes en cada etapa, el total es $a_1 a_2 \cdots a_k$.

5.3 Permutaciones y arreglos

Situación	Cantidad
Ordenar n objetos distintos	$n!$
Elegir y ordenar k de n objetos	$P(n,k) = \dfrac{n!}{(n-k)!}$
Elegir y ordenar k posiciones, con repetición	n^k
Ordenar n objetos con repeticiones de tamaños $n_1, \ldots, n_t$	$\dfrac{n!}{n_1! \cdots n_t!}$
Permutaciones circulares de n objetos distintos	$(n-1)!$

En permutaciones circulares se consideran iguales las rotaciones. Si también se consideran iguales las reflexiones, el conteo puede cambiar y debe analizarse según la simetría del problema.

5.4 Combinaciones

Elegir k objetos de un conjunto de n objetos distintos, sin importar el orden:

$$\boxed{\binom{n}{k} = \frac{n!}{k!(n-k)!}}.$$

Propiedades:

$$\begin{aligned}
\binom{n}{0} &= \binom{n}{n} = 1, \\
\binom{n}{k} &= \binom{n}{n-k}, \\
\binom{n}{k} &= \binom{n-1}{k} + \binom{n-1}{k-1} \quad \text{(Pascal)}, \\
\sum_{k=0}^{n} \binom{n}{k} &= 2^n, \\
\sum_{k=0}^{n} k\binom{n}{k} &= n2^{n-1}.
\end{aligned}$$

5.5 Combinaciones con repetición

Elegir k elementos de n tipos, permitiendo repetir y sin importar el orden:

$$\boxed{\binom{n+k-1}{k}}.$$

Es equivalente a contar soluciones enteras no negativas de $x_1 + \cdots + x_n = k$.

5.6 Estrellas y barras

Número de soluciones enteras no negativas:

$$x_1 + x_2 + \cdots + x_k = n, \qquad x_i \geq 0 : \quad \boxed{\binom{n+k-1}{k-1}}.$$

Número de soluciones enteras positivas, $x_i \geq 1$:

$$\boxed{\binom{n-1}{k-1}}.$$

Si $x_i \geq a_i$, se hace el cambio $y_i = x_i - a_i$ y se resuelve con las nuevas variables no negativas.

Idea para reconocer el problema

Palabras como repartir", "distribuir", "poner objetos en cajas.º "soluciones enteras de una suma"suelen sugerir estrellas y barras. Confirmar siempre si las cajas/personas son distinguibles y si se permite que alguna reciba cero.

5.7 Coeficiente multinomial

Si se distribuyen n posiciones en grupos de tamaños $n_1, \ldots, n_k$ con suma n:

$$\boxed{\binom{n}{n_1, n_2, \ldots, n_k} = \frac{n!}{n_1! n_2! \cdots n_k!}}.$$

Es el coeficiente de $x_1^{n_1} \cdots x_k^{n_k}$ en $(x_1 + \cdots + x_k)^n$.

5.8 Teorema del binomio

$$(a+b)^n = \sum_{k=0}^{n} \binom{n}{k} a^{n-k} b^k.$$

Identidades derivadas:

$$\sum_{k=0}^{n} (-1)^k \binom{n}{k} = 0 \quad (n > 0),$$

$$\sum_{k=0}^{n} \binom{n}{k} x^k = (1+x)^n,$$

$$\sum_{k=0}^{n} \binom{n}{k}^2 = \binom{2n}{n}.$$

En la última igualdad, $\binom{n}{k}^2$ significa $\left(\binom{n}{k}\right)^2$.

5.9 Identidades de Vandermonde y hockey-stick

$$\sum_{i=0}^{k} \binom{r}{i} \binom{s}{k-i} = \binom{r+s}{k}.$$

$$\sum_{i=r}^{n} \binom{i}{r} = \binom{n+1}{r+1}.$$

La segunda se conoce como identidad del palo de hockey y es útil cuando se suman combinaciones con límite superior variable.

5.10 Principio de inclusión-exclusión

Para dos conjuntos:

$$|A \cup B| = |A| + |B| - |A \cap B|.$$

Para tres:

$$\begin{aligned} |A \cup B \cup C| = {} & |A| + |B| + |C| \\ & - |A \cap B| - |A \cap C| - |B \cap C| \\ & + |A \cap B \cap C|. \end{aligned}$$

En general:

$$\left|\bigcup_{i=1}^{n} A_i\right| = \sum_{\emptyset \neq S \subseteq \{1,\ldots,n\}} (-1)^{|S|+1} \left|\bigcap_{i \in S} A_i\right|.$$

Para contar enteros de 1 a N divisibles por al menos uno de $a_1, \ldots, a_k$, se combinan múltiplos y MCM:

$$|A_i| = \lfloor N/a_i \rfloor, \qquad |A_i \cap A_j| = \lfloor N/\operatorname{mcm}(a_i, a_j) \rfloor.$$

La fórmula general requiere $2^k - 1$ subconjuntos, así que suele ser viable cuando k es pequeño.

5.11 Lema de Burnside

Cuando varias configuraciones se consideran iguales por rotaciones, reflexiones u otras simetrías, se pueden contar las clases de equivalencia con:

$$\boxed{\#\text{ órbitas} = \frac{1}{|G|} \sum_{g \in G} |\operatorname{Fix}(g)|},$$

donde G es el grupo de simetrías y $\operatorname{Fix}(g)$ es el conjunto de configuraciones que no cambian al aplicar g. Es especialmente útil para collares, pulseras, coloreos circulares y tableros simétricos.

Ojo en competencia

En aritmética modular, la división por $|G|$ exige que $|G|$ tenga inverso módulo M. Si no lo tiene, no se puede dividir simplemente por el tamaño del grupo.

5.12 Desarreglos

Un desarreglo es una permutación en la que ningún elemento queda en su posición original. Su cantidad es:

$$\boxed{!n = n! \sum_{k=0}^{n} \frac{(-1)^k}{k!}}.$$

Recurrencia:

$$!0 = 1, \quad !1 = 0, \quad !n = (n-1)(!(n-1) + !(n-2)).$$

La probabilidad de que una permutación aleatoria sea un desarreglo se aproxima a $1/e$ cuando n crece.

5.13 Números de Catalan

$$\boxed{C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1}}.$$

Recurrencia:

$$C_0 = 1, \qquad C_{n+1} = \sum_{i=0}^{n} C_i C_{n-i}.$$

Primeros valores: $1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, \ldots$

Interpretaciones frecuentes:

- Caminos de Dyck de longitud $2n$ que nunca bajan de la altura cero.

- Formas de parentizar correctamente $n+1$ factores.
- Árboles binarios completos con n nodos internos (según la convención de conteo).
- Triangulaciones de un polígono convexo de $n+2$ lados.
- Secuencias de n aperturas y n cierres de paréntesis bien balanceadas.

5.14 Números de Stirling

5.14.1 Segunda especie

$S(n,k)$ cuenta particiones de n elementos distinguibles en k grupos no vacíos e indistinguibles:

$$S(n,k) = S(n-1,k-1) + kS(n-1,k).$$

Condiciones: $S(0,0)=1$, $S(n,0)=0$ para $n>0$. También:

$$S(n,k) = \frac{1}{k!}\sum_{i=0}^{k}(-1)^{k-i}\binom{k}{i}i^n.$$

5.14.2 Primera especie

Los números de Stirling sin signo de primera especie cuentan permutaciones de n elementos con exactamente k ciclos. Recurrencia:

$$\begin{bmatrix} n \\ k \end{bmatrix} = \begin{bmatrix} n-1 \\ k-1 \end{bmatrix} + (n-1)\begin{bmatrix} n-1 \\ k \end{bmatrix}.$$

5.15 Números de Bell

B_n cuenta todas las particiones de un conjunto de n elementos:

$$B_n = \sum_{k=0}^{n} S(n,k).$$

Recurrencia de Bell:

$$B_{n+1} = \sum_{k=0}^{n}\binom{n}{k}B_k, \qquad B_0 = 1.$$

5.16 Particiones de enteros

$p(n)$ cuenta las formas de escribir n como suma de enteros positivos sin considerar el orden. Función generadora:

$$\sum_{n=0}^{\infty} p(n)x^n = \prod_{k=1}^{\infty}\frac{1}{1-x^k}.$$

No confundir particiones de un entero con particiones de un conjunto: son problemas diferentes.

5.17 Conteos pequeños muy comunes

$$\begin{aligned}
\text{Subconjuntos de un conjunto de } n \text{ elementos: } & 2^n, \\
\text{Subconjuntos de tamaño } k: & \binom{n}{k}, \\
\text{Cadenas binarias de longitud } n: & 2^n, \\
\text{Cadenas de longitud } n \text{ sobre alfabeto de } q \text{ símbolos}: & q^n, \\
\text{Cadenas binarias sin dos unos consecutivos}: & F_{n+2}.
\end{aligned}$$

En la última expresión $F_0 = 0, F_1 = 1$ son números de Fibonacci.

Capítulo 6

Funciones generadoras y convoluciones

6.1 Idea de función generadora

A una secuencia $a_0, a_1, a_2, \ldots$ se le asocia la función generadora ordinaria:

$$A(x) = \sum_{n \geq 0} a_n x^n.$$

El coeficiente de x^n guarda la información del término a_n. Multiplicar funciones generadoras corresponde a combinar elecciones y realizar una convolución de coeficientes.

6.2 Funciones generadoras básicas

Para $|x| < 1$:

$$\begin{aligned}
\frac{1}{1-x} &= \sum_{n \geq 0} x^n, \\
\frac{1}{(1-x)^2} &= \sum_{n \geq 0} (n+1) x^n, \\
\frac{1}{(1-x)^k} &= \sum_{n \geq 0} \binom{n+k-1}{k-1} x^n, \\
\frac{1}{1-ax} &= \sum_{n \geq 0} a^n x^n.
\end{aligned}$$

La identidad del binomio generalizado es:

$$(1-x)^{-r} = \sum_{n \geq 0} \binom{n+r-1}{r-1} x^n \quad (r \in \mathbb{N},\ r \geq 1).$$

6.3 Convolución

Si $C(x) = A(x)B(x)$, entonces:

$$c_n = \sum_{i=0}^{n} a_i b_{n-i}.$$

La multiplicación directa cuesta $\mathcal{O}(n^2)$. Para secuencias largas se puede acelerar con FFT o NTT, según el dominio numérico.

Idea para reconocer el problema

Cuando un problema pide elegir una cantidad de cada tipo, combinar dos secuencias de maneras independientes o multiplicar polinomios, puede haber una convolución escondida.

Capítulo 7

Probabilidad y valor esperado

7.1 Probabilidad básica

Para un espacio muestral finito equiprobable:

$$\mathbb{P}(A) = \frac{|A|}{|\Omega|}, \qquad 0 \leq \mathbb{P}(A) \leq 1.$$

Complemento:

$$\mathbb{P}(\overline{A}) = 1 - \mathbb{P}(A).$$

Unión de eventos:

$$\mathbb{P}(A \cup B) = \mathbb{P}(A) + \mathbb{P}(B) - \mathbb{P}(A \cap B).$$

Si son independientes:

$$\mathbb{P}(A \cap B) = \mathbb{P}(A)\mathbb{P}(B).$$

Condicional:

$$\mathbb{P}(A \mid B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)}, \quad \mathbb{P}(B) > 0.$$

Bayes:

$$\mathbb{P}(A \mid B) = \frac{\mathbb{P}(B \mid A)\mathbb{P}(A)}{\mathbb{P}(B)}.$$

7.2 Variables aleatorias discretas

Una variable aleatoria X asigna un número a cada resultado. Su función de masa de probabilidad es $p_X(x) = \mathbb{P}(X = x)$ y cumple $\sum_x p_X(x) = 1$. Su función de distribución acumulada es $F_X(t) = \mathbb{P}(X \leq t)$. Su esperanza es:

$$\boxed{\mathbb{E}[X] = \sum_x x\mathbb{P}(X = x)}.$$

Si g es una función:

$$\mathbb{E}[g(X)] = \sum_x g(x)\mathbb{P}(X = x).$$

Linealidad de la esperanza:

$$\boxed{\mathbb{E}[aX + bY + c] = a\mathbb{E}[X] + b\mathbb{E}[Y] + c.}$$

La linealidad se cumple incluso si X e Y no son independientes.

7.3 Variables indicadoras

Para un evento A, define $I_A = 1$ si ocurre y 0 en caso contrario. Entonces:

$$\mathbb{E}[I_A] = \mathbb{P}(A).$$

Si X cuenta cuántos eventos A_i ocurren:

$$X = \sum_i I_{A_i} \quad \Longrightarrow \quad \mathbb{E}[X] = \sum_i \mathbb{P}(A_i).$$

Esta técnica permite calcular esperanzas sin conocer toda la distribución de X.

7.4 Varianza

$$\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2.$$

Desviación estándar: $\sigma_X = \sqrt{\mathrm{Var}(X)}$. Para variables independientes:

$$\mathrm{Var}(X + Y) = \mathrm{Var}(X) + \mathrm{Var}(Y).$$

En general:

$$\mathrm{Var}(X + Y) = \mathrm{Var}(X) + \mathrm{Var}(Y) + 2\,\mathrm{Cov}(X, Y).$$

7.5 Distribuciones discretas comunes

Distribución	**Probabilidad**	**Esperanza y varianza**
Bernoulli(p)	$P(X=1)=p,\ P(X=0)=1-p$	$E[X]=p,\ \mathrm{Var}(X)=p(1-p)$
Binomial(n,p)	$P(X=k)=\binom{n}{k}p^k(1-p)^{n-k}$	$E[X]=np,\ \mathrm{Var}(X)=np(1-p)$
Geométrica(p)	$P(X=k)=(1-p)^{k-1}p,\ k\geq 1$	$E[X]=1/p,\ \mathrm{Var}(X)=(1-p)/p^2$
Hipergeométrica (N,K,n)	$P(X=k)=\dfrac{\binom{K}{k}\binom{N-K}{n-k}}{\binom{N}{n}}$	$p=K/N;\ E[X]=np;\ \mathrm{Var}(X)=np(1-p)\frac{N-n}{N-1}$
Poisson(λ)	$P(X=k)=e^{-\lambda}\lambda^k/k!$	$E[X]=\lambda,\ \mathrm{Var}(X)=\lambda$

Lectura de parámetros:

- Binomial: n ensayos independientes con probabilidad de éxito p.
- Geométrica: número de ensayos hasta el primer éxito.
- Hipergeométrica: extracción sin reemplazo de una población con K éxitos entre N elementos.
- Poisson: conteo de eventos con tasa media λ bajo sus hipótesis usuales.

7.6 Problemas clásicos de esperanza

7.6.1 Número esperado de éxitos

En n pruebas, si cada una tiene probabilidad p_i de éxito, la cantidad esperada de éxitos es:

$$\mathbb{E}[X] = \sum_{i=1}^{n} p_i.$$

No hace falta que las pruebas sean independientes.

7.6.2 Coleccionista de cupones

Si hay n tipos equiprobables y cada intento entrega uno, el número esperado de intentos para conseguir todos es:

$$\mathbb{E}[T] = nH_n = n\left(1 + \frac{1}{2} + \cdots + \frac{1}{n}\right) \approx n \ln n + \gamma n.$$

7.6.3 Tiempo de espera geométrico

Si cada intento independiente tiene probabilidad p de éxito, el número esperado de intentos hasta el primer éxito es $1/p$.

Ojo en competencia

No multiplicar probabilidades si los eventos no son independientes. Tampoco confundir "hasta el primer éxito"(geométrica con soporte desde 1) con "número de fallos antes del éxito"(soporte desde 0).

Capítulo 8

Sucesiones, recurrencias y matrices

8.1 Fibonacci

$$F_0 = 0, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2}.$$

Fórmula cerrada (útil teóricamente, no siempre para calcular con enteros exactos):

$$F_n = \frac{\phi^n - \psi^n}{\sqrt{5}}, \quad \phi = \frac{1+\sqrt{5}}{2}, \quad \psi = \frac{1-\sqrt{5}}{2}.$$

Identidades:

$$F_{n+1}F_{n-1} - F_n^2 = (-1)^n,$$
$$\sum_{i=0}^{n} F_i = F_{n+2} - 1,$$
$$\sum_{i=1}^{n} F_i^2 = F_n F_{n+1}.$$

8.2 Recurrencias lineales

Una recurrencia lineal homogénea de orden k tiene la forma:

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k}.$$

Su ecuación característica es:

$$r^k - c_1 r^{k-1} - c_2 r^{k-2} - \cdots - c_k = 0.$$

Con raíces distintas r_i, la solución suele escribirse como combinación de potencias r_i^n. Si hay raíces repetidas, aparecen factores polinómicos en n.

8.3 Multiplicación de matrices

Para matrices compatibles:

$$(AB)_{ij} = \sum_k A_{ik} B_{kj}.$$

La multiplicación de matrices $n \times n$ por el método clásico cuesta $\mathcal{O}(n^3)$. La potenciación binaria de matrices calcula A^k en $\mathcal{O}(n^3 \log k)$.

8.4 Fibonacci con matrices

$$\begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

Se puede usar la misma técnica para cualquier recurrencia lineal de orden pequeño, construyendo un vector de estado con los últimos términos.

8.5 Fast doubling para Fibonacci

Identidades:

$$F_{2k} = F_k(2F_{k+1} - F_k),$$
$$F_{2k+1} = F_k^2 + F_{k+1}^2.$$

Permiten obtener (F_n, F_{n+1}) en $\mathcal{O}(\log n)$.

```
pair<long long, long long> fib(long long n, long long mod) {
    if (n == 0) return {0, 1};
    auto [a, b] = fib(n / 2, mod);
    long long c = (long long)((__int128)a * ((2*b % mod - a + mod) % mod) % mod);
    long long d = (long long)(((__int128)a*a + (__int128)b*b) % mod);
    if (n & 1) return {d, (c + d) % mod};
    return {c, d};
}
```

8.6 Diferencias finitas

Para una secuencia a_n, su primera diferencia es $\Delta a_n = a_{n+1} - a_n$. Si las diferencias de orden k son constantes, la secuencia proviene de un polinomio de grado k. Esto ayuda a detectar fórmulas polinómicas en secuencias pequeñas.

Capítulo 9

Combinatoria modular y precálculos

9.1 Factoriales e inversos factoriales

Si p es primo y $0 \le n < p$, definimos:

$$fact[n] = n! \bmod p, \qquad invfact[n] = (n!)^{-1} \bmod p.$$

Entonces:

$$\boxed{\binom{n}{k} \bmod p = fact[n]\, invfact[k]\, invfact[n-k] \bmod p.}$$

Se calcula $fact$ de izquierda a derecha, un inverso factorial desde el final con Fermat, y luego los demás hacia atrás.

```
const long long MOD = 1'000'000'007;
vector<long long> fact, invfact;

long long modpow(long long a, long long e, long long mod);

void initComb(int n) {
    fact.assign(n + 1, 1);
    invfact.assign(n + 1, 1);
    for (int i = 1; i <= n; ++i)
        fact[i] = fact[i - 1] * i % MOD;
    invfact[n] = modpow(fact[n], MOD - 2, MOD);
    for (int i = n; i >= 1; --i)
        invfact[i - 1] = invfact[i] * i % MOD;
}

long long C(int n, int k) {
    if (k < 0 || k > n) return 0;
    return fact[n] * invfact[k] % MOD * invfact[n-k] % MOD;
}
```

Ojo en competencia

Esta plantilla requiere que `MOD` sea primo y que el mayor factorial precalculado cumpla $n < MOD$. Si $n \ge p$, los factoriales son cero módulo p y esta técnica directa deja de servir. Revisar Lucas, factoriales sin factores de p o métodos específicos del problema.

9.2 Combinaciones con k pequeño

Para $k \leq n$:

$$\binom{n}{k} = \prod_{i=1}^{k} \frac{n-k+i}{i}.$$

Como enteros exactos, se puede reducir numerador y denominador con MCD en cada paso. En módulo, cada división exige que el denominador sea invertible. La versión multiplicativa puede ser útil cuando solo hay una consulta y k es pequeño.

9.3 Triángulo de Pascal

La recurrencia:

$$C[n][k] = C[n-1][k-1] + C[n-1][k]$$

permite calcular combinaciones sin inversos modulares, con $\mathcal{O}(n^2)$ tiempo y memoria (o $\mathcal{O}(n)$ memoria si se actualiza de derecha a izquierda).

9.4 Combinaciones cuando el módulo no es primo

Si el módulo es compuesto, no se puede usar automáticamente a^{M-2} como inverso ni dividir por factoriales. Alternativas según los límites:

- Pascal, si n es pequeño.
- Factorización del módulo y CRT.
- Métodos para binomiales módulo potencias de primos.
- Factorizar numeradores y denominadores y controlar exponentes primos.

La solución correcta depende de n, k y de la estructura del módulo.

Capítulo 10

Álgebra lineal

10.1 Vectores y producto punto

Para $u = (u_1, \ldots, u_n)$ y $v = (v_1, \ldots, v_n)$:

$$u \cdot v = \sum_{i=1}^{n} u_i v_i.$$

En dos dimensiones, si los vectores son no nulos:

$$u \cdot v = \|u\| \, \|v\| \cos\theta.$$

Por tanto, $u \cdot v = 0$ indica perpendicularidad.

10.2 Determinante

Para una matriz 2×2:

$$\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc.$$

Para una matriz 3×3, se puede desarrollar por cofactores o usar reducción. Propiedades:

$$\begin{gathered} \det(AB) = \det(A)\det(B), \\ \det(A^T) = \det(A), \\ A \text{ es invertible} \iff \det(A) \neq 0. \end{gathered}$$

10.3 Sistemas lineales

Un sistema $Ax = b$ puede tener:

- ninguna solución;
- una única solución;
- infinitas soluciones.

La eliminación de Gauss transforma el sistema mediante operaciones elementales de fila. Para una matriz $n \times n$, el método clásico cuesta $\mathcal{O}(n^3)$.

Ojo en competencia

En aritmética modular, la eliminación de Gauss requiere dividir por pivotes no nulos e invertibles. En un campo $\mathbb{F}_p$ esto equivale a que el pivote no sea cero. En módulo compuesto, un pivote no nulo puede no tener inverso.

10.4 Rango

El rango de una matriz es la cantidad máxima de filas (o columnas) linealmente independientes. Un sistema compatible cumple:

$$\operatorname{rank}(A) = \operatorname{rank}(A \mid b).$$

Tiene solución única si además ese rango coincide con el número de incógnitas.

Capítulo 11

Geometría para programación competitiva

11.1 Puntos y vectores

Sean $A = (x_1, y_1)$ y $B = (x_2, y_2)$:

$$\overrightarrow{AB} = (x_2 - x_1, y_2 - y_1).$$

Distancia al cuadrado:

$$d^2(A, B) = (x_2 - x_1)^2 + (y_2 - y_1)^2.$$

Comparar distancias al cuadrado evita calcular raíces cuando solo se necesita saber cuál es mayor.

11.2 Producto punto y producto cruzado

Para $u = (u_x, u_y)$ y $v = (v_x, v_y)$:

$$u \cdot v = u_x v_x + u_y v_y, \qquad u \times v = u_x v_y - u_y v_x.$$

El producto cruzado en 2D es un escalar:

- $u \times v > 0$: giro antihorario de u a v.
- $u \times v < 0$: giro horario.
- $u \times v = 0$: vectores colineales.

11.3 Orientación de tres puntos

$$\text{orient}(A, B, C) = (B - A) \times (C - A).$$

El signo indica si C queda a la izquierda, a la derecha o sobre la recta orientada AB. Con coordenadas enteras, usar `long long` o `__int128` para evitar overflow.

11.4 Área

Área de triángulo:

$$A_{\triangle ABC} = \frac{1}{2} \left| (B - A) \times (C - A) \right|.$$

Fórmula del zapatero para un polígono de vértices ordenados (x_i, y_i):

$$\boxed{2A = \left| \sum_{i=0}^{n-1} (x_i y_{i+1} - y_i x_{i+1}) \right|},$$

donde el índice se toma módulo n.

11.5 Rectas

Forma general:

$$ax + by + c = 0.$$

Distancia del punto (x_0, y_0) a la recta:

$$d = \frac{|ax_0 + by_0 + c|}{\sqrt{a^2 + b^2}}.$$

Dos rectas $a_1x+b_1y+c_1 = 0$ y $a_2x+b_2y+c_2 = 0$ son paralelas cuando sus normales son proporcionales: $a_1b_2 - a_2b_1 = 0$.

11.6 Proyección de un punto sobre una recta

La proyección de P sobre la recta que pasa por A con dirección v es:

$$Q = A + v\frac{(P - A) \cdot v}{v \cdot v}.$$

Para el segmento AB, se debe limitar el parámetro:

$$t = \text{máx}\left(0, \text{mín}\left(1, \frac{(P - A) \cdot (B - A)}{\|B - A\|^2}\right)\right), \quad Q = A + t(B - A).$$

11.7 Círculo

Área y perímetro:

$$A = \pi r^2, \qquad C = 2\pi r.$$

Ecuación con centro (h, k):

$$(x - h)^2 + (y - k)^2 = r^2.$$

Distancia entre centros d y radios r_1, r_2:

- $d > r_1 + r_2$: círculos separados.
- $d = r_1 + r_2$: tangentes exteriormente.
- $|r_1 - r_2| < d < r_1 + r_2$: dos intersecciones.
- $d = |r_1 - r_2|$: tangentes interiormente (si $d > 0$).
- $d < |r_1 - r_2|$: uno dentro del otro sin tocarse.

11.8 Puntos de coordenadas enteras

En un segmento entre $A = (x_1, y_1)$ y $B = (x_2, y_2)$ con extremos enteros, la cantidad de puntos de coordenadas enteras sobre el segmento, incluyendo extremos, es:

$$\boxed{\gcd(|x_2 - x_1|, |y_2 - y_1|) + 1}.$$

Para un polígono simple cuyos vértices tienen coordenadas enteras, la cantidad de puntos de la frontera es:

$$B = \sum_{i=0}^{n-1} \gcd(|x_{i+1} - x_i|, |y_{i+1} - y_i|).$$

11.9 Teorema de Pick

Para un polígono simple, no degenerado, cuyos vértices están en la retícula:

$$\boxed{A = I + \frac{B}{2} - 1},$$

donde I es la cantidad de puntos de retícula interiores y B los puntos de retícula en la frontera.

11.10 Transformaciones

Rotación de (x, y) alrededor del origen:

$$\begin{aligned} 90^\circ \text{ antihorario}: &\quad (-y, x), \\ 180^\circ: &\quad (-x, -y), \\ 270^\circ \text{ antihorario}: &\quad (y, -x). \end{aligned}$$

Reflexiones:

$$\text{eje } x : (x, -y), \quad \text{eje } y : (-x, y), \quad \text{recta } y = x : (y, x).$$

Ojo en competencia

Los errores geométricos más comunes son: usar `double` para comparaciones exactas que pueden resolverse con enteros; no contemplar puntos colineales; olvidar que un segmento incluye o excluye extremos según el enunciado; y desbordar al multiplicar coordenadas.

Capítulo 12

Bitmasks, XOR y juegos

12.1 Operaciones de bits

Para un entero no negativo `mask`:

$$\begin{aligned}
\text{consultar bit } i &: (mask \gg i)\ \&\ 1,\\
\text{activar bit } i &: mask \mid (1 \ll i),\\
\text{apagar bit } i &: mask\ \&\ \sim(1 \ll i),\\
\text{invertir bit } i &: mask \hat{}\ (1 \ll i),\\
\text{quitar bit menos significativo activo} &: mask\ \&\ (mask - 1).
\end{aligned}$$

Número de subconjuntos: 2^n. Máscara del conjunto completo de n bits: $(1 \ll n) - 1$ (si el desplazamiento es válido para el tipo).

12.2 Enumerar submáscaras

Para enumerar todas las submáscaras no nulas de `mask`:

```
for (int sub = mask; sub; sub = (sub - 1) & mask) {
    // procesar sub
}
```

Para incluir la submáscara cero, procesarla al final. Enumerar todos los pares $(sub, mask)$ con $sub \subseteq mask$ cuesta $\mathcal{O}(3^n)$ en total.

12.3 XOR

Propiedades:

$$a \oplus a = 0, \quad a \oplus 0 = a, \quad a \oplus b = b \oplus a, \quad (a \oplus b) \oplus c = a \oplus (b \oplus c).$$

El XOR de un arreglo puede construirse con prefijos:

$$p_0 = 0, \quad p_i = a_1 \oplus \cdots \oplus a_i, \quad xor(l, r) = p_r \oplus p_{l-1}.$$

12.4 Nim

En Nim normal hay varios montones; en cada turno se retiran una o más piedras de un solo montón. La posición es perdedora para quien juega si y solo si:

$$\boxed{a_1 \oplus a_2 \oplus \cdots \oplus a_n = 0.}$$

Si el XOR es distinto de cero, existe un movimiento que lo convierte en cero.

12.5 Grundy y mex

Para un estado de juego imparcial acíclico:

$$g(s) = \text{mex}\{g(t) : t \text{ es un estado alcanzable desde } s\}.$$

`mex` es el menor entero no negativo que no aparece en el conjunto. Una posición es perdedora si su Grundy es cero. La suma de juegos independientes se representa con XOR de sus Grundy:

$$g(A + B) = g(A) \oplus g(B).$$

Capítulo 13

Transformadas y multiplicación rápida de polinomios

13.1 Convolución discreta

Para dos arreglos a y b de longitudes n y m, su convolución es:

$$c_k = \sum_{i+j=k} a_i b_j,$$

y tiene longitud $n + m - 1$. El método directo cuesta $\mathcal{O}(nm)$.

13.2 Transformada discreta de Fourier (DFT)

Para una secuencia de longitud N:

$$A_k = \sum_{j=0}^{N-1} a_j \omega_N^{jk}, \quad \omega_N = e^{-2\pi i/N}.$$

La transformada inversa es:

$$a_j = \frac{1}{N} \sum_{k=0}^{N-1} A_k \omega_N^{-jk}.$$

La FFT calcula la DFT en $\mathcal{O}(N \log N)$ cuando N se elige adecuadamente. Para convolución, se rellena con ceros hasta una longitud suficiente, se transforman ambos arreglos, se multiplican punto a punto y se aplica la inversa.

13.3 NTT

La transformada rápida número-teórica (NTT) reemplaza raíces complejas por raíces de la unidad módulo un primo adecuado. Un módulo clásico es $998244353 = 119 \cdot 2^{23} + 1$, que admite longitudes potencia de dos hasta 2^{23} con una raíz primitiva apropiada.

Ojo en competencia

La NTT no funciona con cualquier módulo ni cualquier longitud. Hay que verificar que el módulo sea primo, que la longitud divida a $p - 1$ y que se use una raíz de la unidad de orden exacto requerido. Para coeficientes grandes puede ser necesario usar varios módulos y reconstrucción CRT.

Capítulo 14

Técnicas numéricas y seguridad de implementación

14.1 Tipos enteros

Tipo	Tamaño típico	Uso
`int`	32 bits	Valores de hasta aproximadamente $2 \cdot 10^9$ en magnitud.
`long long`	64 bits	Conteos y productos moderados; hasta aproximadamente $9.22 \cdot 10^{18}$.
`__int128`	128 bits	Productos intermedios grandes en GCC/Clang; no es estándar ISO C++.
`double`	64 bits	Cálculo aproximado; cerca de 15-16 dígitos decimales de precisión.

14.2 Raíz cuadrada entera

Para verificar si $d^2 \leq n$ sin overflow, usar `1LL * d * d <= n` cuando los límites lo permitan. Para un `n` de 64 bits, se puede comenzar con `sqrtl` y corregir el resultado mediante productos en `__int128`.

14.3 Comparaciones de fracciones

Para comparar a/b y c/d, con denominadores positivos, se puede comparar ad y bc. Si el producto puede desbordar, usar `__int128` o una comparación exacta por fracciones continuas.

14.4 Reducción de fracciones

$$\frac{a}{b} = \frac{a/g}{b/g}, \quad g = \gcd(|a|, |b|), \quad b > 0.$$

Normalizar el denominador positivo facilita ordenar y comparar fracciones.

14.5 Errores frecuentes de precisión

- No comparar flotantes con `==` si el resultado proviene de operaciones aproximadas.
- Para igualdad aproximada, elegir una tolerancia coherente con la escala del problema.

- No usar `double` cuando la respuesta debe ser exacta y hay una alternativa entera.
- Cuidar la acumulación de errores al sumar muchos valores muy distintos.

Capítulo 15

Checklist de resolución en competencia

Antes de programar, el equipo debería poder responder:

1. ¿Qué se pide exactamente: contar, optimizar, decidir, construir o calcular una probabilidad?
2. ¿Qué representan las variables? ¿Cuáles son sus límites?
3. ¿Importa el orden? ¿Se permite repetir? ¿Los objetos son distinguibles?
4. ¿La respuesta es exacta o módulo M? Si es modular, ¿M es primo?
5. ¿Una división modular necesita inverso? ¿Existe?
6. ¿Qué puede desbordarse: productos, factoriales, áreas, sumas o índices?
7. ¿La complejidad estimada entra en el límite de tiempo y memoria?
8. ¿Qué casos extremos deben probarse: cero, uno, máximo, repetidos, negativos, colineales o sin solución?

15.1 Complejidades orientativas

Complejidad	Tamaño que suele ser razonable (muy aproximado)
$\mathcal{O}(1)$	Cualquier tamaño práctico.
$\mathcal{O}(\log n)$	Muy grande; depende de la constante y del número de consultas.
$\mathcal{O}(\sqrt{n})$	Hasta valores grandes, según implementación.
$\mathcal{O}(n)$	Habitualmente hasta 10^7–10^8 operaciones simples, dependiendo del juez.
$\mathcal{O}(n \log n)$	Frecuentemente adecuado para 10^5–10^6.
$\mathcal{O}(n^2)$	A menudo n alrededor de 10^3; depende de las constantes.
$\mathcal{O}(n^3)$	A menudo n alrededor de 200–500 con optimización y límites favorables.
$\mathcal{O}(2^n)$	Generalmente para n pequeño, aproximadamente hasta 20–25.
$\mathcal{O}(3^n)$	Para n todavía más pequeño.

Estas cifras no son límites universales. El lenguaje, hardware, memoria, constantes, estructura de los datos y número de casos pueden cambiar bastante el resultado.

15.2 Casos de prueba matemáticos

Al crear pruebas, no usar solamente ejemplos aleatorios. Agregar:

- mínimos: $n = 0$, $n = 1$, conjunto vacío, una sola posición;
- simetrías: $k = 0$, $k = n$, $k = n/2$;

- divisibilidad: coprimos, uno divide al otro, MCD mayor que uno;
- módulo: módulo primo, compuesto y factores comunes con el denominador;
- geometría: puntos iguales, colineales, orientación cero y coordenadas extremas;
- combinatoria: ninguna forma válida, una forma y muchas formas.

Capítulo A

Plantillas C++17 de apoyo

A.1 MCD y MCM

```cpp
long long gcdll(long long a, long long b) {
    while (b != 0) {
        long long r = a % b;
        a = b;
        b = r;
    }
    return a >= 0 ? a : -a;
}

long long lcmll(long long a, long long b) {
    if (a == 0 || b == 0) return 0;
    long long g = gcdll(a, b);
    __int128 z = (__int128)(a / g) * b; // revisar si excede long long
    if (z < 0) z = -z;
    return (long long)z;
}
```

A.2 Menor factor primo (SPF) con criba lineal

```cpp
vector<int> buildSPF(int n) {
    vector<int> spf(n + 1), primes;
    for (int i = 2; i <= n; ++i) {
        if (spf[i] == 0) {
            spf[i] = i;
            primes.push_back(i);
        }
        for (int p : primes) {
            if (p > spf[i] || 1LL * p * i > n) break;
            spf[p * i] = p;
        }
    }
    return spf;
}

vector<pair<int,int>> factorize(int x, const vector<int>& spf) {
    vector<pair<int,int>> f;
    while (x > 1) {
        int p = spf[x], cnt = 0;
        while (x % p == 0) x /= p, ++cnt;
        f.push_back({p, cnt});
```

```
    }
    return f;
}
```

A.3 Función phi con factorización

```
long long phi(long long n) {
    long long ans = n;
    for (long long p = 2; p <= n / p; ++p) {
        if (n % p == 0) {
            while (n % p == 0) n /= p;
            ans -= ans / p;
        }
    }
    if (n > 1) ans -= ans / n;
    return ans;
}
```

A.4 Potenciación de matrices

```
using Matrix = vector<vector<long long>>;

Matrix multiply(const Matrix& A, const Matrix& B, long long mod) {
    int n = (int)A.size();
    int m = (int)B[0].size();
    int k = (int)B.size();
    Matrix C(n, vector<long long>(m, 0));
    for (int i = 0; i < n; ++i)
        for (int t = 0; t < k; ++t)
            for (int j = 0; j < m; ++j)
                C[i][j] = (C[i][j] +
                    (__int128)A[i][t] * B[t][j]) % mod;
    return C;
}

Matrix matrixPower(Matrix A, long long e, long long mod) {
    int n = (int)A.size();
    Matrix R(n, vector<long long>(n, 0));
    for (int i = 0; i < n; ++i) R[i][i] = 1;
    while (e > 0) {
        if (e & 1) R = multiply(R, A, mod);
        A = multiply(A, A, mod);
        e >>= 1;
    }
    return R;
}
```

A.5 Orientación geométrica entera

```
struct Point {
    long long x, y;
};

```

```
__int128 cross(Point a, Point b, Point c) {
    return ((__int128)b.x - a.x) * ((__int128)c.y - a.y)
         - ((__int128)b.y - a.y) * ((__int128)c.x - a.x);
}

int orientation(Point a, Point b, Point c) {
    __int128 z = cross(a, b, c);
    if (z > 0) return 1;
    if (z < 0) return -1;
    return 0;
}
```

A.6 Plantilla mínima de competencia

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using i128 = __int128_t;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    // Leer entrada y resolver.
    return 0;
}
```

Capítulo B

Glosario breve

Adyacente Dos estados o vértices unidos por una transición o arista.

Binomial Coeficiente $\binom{n}{k}$, que cuenta elecciones de k elementos entre n.

Congruencia Igualdad de restos módulo un entero.

Convolución Combinación de dos secuencias mediante $c_n = \sum_i a_i b_{n-i}$.

coprimos Dos enteros cuyo MCD es uno.

Factorial $n! = 1 \cdot 2 \cdots n$, con $0! = 1$.

Invariante Propiedad que se mantiene durante cada paso de un procedimiento.

Inverso modular Número que multiplicado por otro da uno módulo m.

Primo Entero mayor que uno con exactamente dos divisores positivos.

Recurrencia Regla que define términos a partir de términos anteriores.

Retícula Conjunto de puntos con coordenadas enteras en el plano.

Submáscara Máscara cuyos bits activos también están activos en otra máscara.

Capítulo C

Referencias y lecturas para ampliar

1. Steven Halim y Felix Halim, *Competitive Programming 4*, volumen 1 y volumen 2. Referencia de organización de temas, técnicas de resolución y enfoque de entrenamiento competitivo.
2. CP-Algorithms, `https://cp-algorithms.com/`. Referencia práctica para algoritmos y estructuras, especialmente teoría de números, aritmética modular, combinatoria, transformadas, álgebra y geometría.
3. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest y Clifford Stein, *Introduction to Algorithms*. Para fundamentos, análisis y demostraciones de algoritmos.
4. Kenneth H. Rosen, *Discrete Mathematics and Its Applications*. Para lógica, conteo, relaciones, recurrencias y probabilidad discreta.

Nota para el equipo

Este teambook es una base inicial de consulta. Antes de una competencia conviene adaptar las plantillas a las reglas del juez, probarlas con casos límite y agregar los métodos que el equipo ya domina. No todas las fórmulas serán útiles en todos los concursos: la habilidad está en reconocer el modelo correcto y justificar que sus condiciones se cumplen.